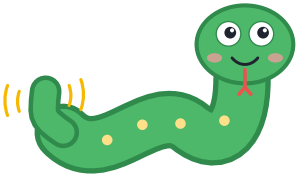
Recitation 2

Part 2: Decisions and Loops

Recitation 2: Decide and Repeat

Choose a road, keep a list, loop and add it up



Dror Jacoby
Tel Aviv University

Where we are


L1
Hello, Python!
L2
Variables & Types
L3
Input & Output
L4
Making Decisions
L5
Lists & Sets
L6
Loops
L7
Functions
L8
Dicts & Debugging
L9
Objects, NumPy & Pandas
L10
Plots & ML
Today we practise L4–L6

Today's plan

◎ By the end of today you can...

- Write programs that pick the right road
- Keep, clean and slice lists, sets and text
- Loop to total, count and find the best



Exercises marked **at home** are for practice after class. 🏠

Topic 1

Recap and warm-up

Recitation 2 • Decisions, lists & loops



TODAY

1. **Recap and warm-up**
2. Worked problems
3. Your turn
4. Exam corner
5. Solutions

Lecture 4 in one slide: decide

```
temp = 31
windy = False
if temp > 30 and not windy:
    print("Beach!")
elif temp > 20 or windy:
    print("Park")
else:
    print("Stay home")
print(bin(5))
```

OUTPUT

```
Beach!
0b101
```

- Top to bottom: the **first True wins**
- = stores, == asks
- **and** both, **or** at least one, **not** flips
- `bin()`: the number in 0s and 1s

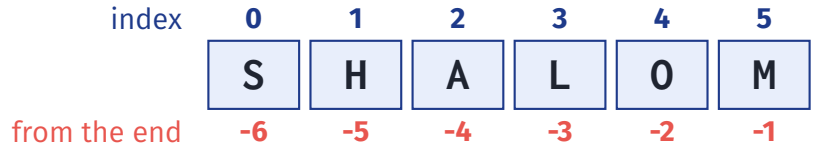
Lecture 5 in one slide: lists and sets

Write	Result
`marks = [80, 95, 70]`	a list, in order
`marks.append(88)`	`[80, 95, 70, 88]`
`marks.insert(0, 60)`	`[60, 80, 95, 70, 88]`
`marks.remove(95)`	`[60, 80, 70, 88]`
`marks.pop()`	gives `88`, leaves `[60, 80, 70]`
`marks.sort()`	`[60, 70, 80]`
`a = {1, 2, 3}, b = {2, 3, 4}`	sets: no duplicates
`a & b, a | b, a - b`	`{2, 3}, {1, 2, 3, 4}, {1}`



Lists keep order. Sets keep each thing once.

Lecture 5 in one slide: positions



`word[0]`, `word[-1]`	`S`, `M`	`word[::2]`	`SAO` (every 2nd)
`word[1:4]`	`HAL` (stop not in)	`word[::-1]`	`MOLAHS` (backwards)
`word[:2]`, `word[3:]`	`SH`, `LOM`	`nums[0] = 99`	lists only, not text

Lecture 6 in one slide: repeat

Write	Means
`while money > 0:`	repeat **as long as** it is True
`for song in playlist:`	one round per item
`for letter in "hey":`	one round per letter
`range(5)`	0, 1, 2, 3, 4
`range(1, 6)`	1, 2, 3, 4, 5
`range(10, 0, -3)`	10, 7, 4, 1
`break`	leave the loop right now



Loop recipes: set up before, update inside, use after.

Warm-up 1: late for class

Predict the output!

```
minutes_late = 12
if minutes_late > 15:
    print("Wait outside")
elif minutes_late > 5:
    print("Sneak in quietly")
elif minutes_late > 0:
    print("Say sorry")
print("Sit down")
```



Warm-up 1: late for class

Predict the output!

```
minutes_late = 12
if minutes_late > 15:
    print("Wait outside")
elif minutes_late > 5:
    print("Sneak in quietly")
elif minutes_late > 0:
    print("Say sorry")
print("Sit down")
```

Answer

```
Sneak in quietly
Sit down
```

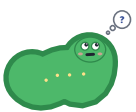
12 > 15 is False, 12 > 5 is True: first True wins. `Sit down` is not indented, so it always runs.



Warm-up 2: and, or, not, bin

Predict the output!

```
hour = 23
weekend = True
print(hour > 22 and weekend)
print(hour < 8 or not weekend)
print(bin(hour))
```



Warm-up 2: and, or, not, bin

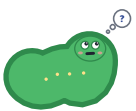
Predict the output!

```
hour = 23
weekend = True
print(hour > 22 and weekend)
print(hour < 8 or not weekend)
print(bin(hour))
```

Answer

```
True
False
0b10111
```

Both sides True, then both sides False. $23 = 16 + 4 + 2 + 1$.



Warm-up 3: the pita bag

Predict the output!

```
bag = ["pita", "hummus"]
bag.append("tahini")
bag.insert(1, "pickles")
last = bag.pop()
print(bag)
print(last, len(bag))
```



Warm-up 3: the pita bag

Predict the output!

```
bag = ["pita", "hummus"]
bag.append("tahini")
bag.insert(1, "pickles")
last = bag.pop()
print(bag)
print(last, len(bag))
```

Answer

```
['pita', 'pickles', 'hummus']
tahini 3
```

Tahini went to the end, so `pop()` took it right back.



Warm-up 4: slice the city

Predict the output!

```
city = "Jerusalem"
print(city[0], city[-1])
print(city[:4])
print(city[4:7])
print(city[::-1])
```

Warm-up 4: slice the city

Predict the output!

```
city = "Jerusalem"
print(city[0], city[-1])
print(city[:4])
print(city[4:7])
print(city[::-1])
```

Answer

```
J m
Jeru
sal
melasureJ
```

Positions 4, 5, 6 are s, a, l. Stop 7 is not in.

Warm-up 5: two loops

Predict the output!

```
total = 0
for i in range(1, 5):
    total = total + i
print(total)
n = 10
while n > 3:
    n = n - 4
print(n)
```



Warm-up 5: two loops

Predict the output!

```
total = 0
for i in range(1, 5):
    total = total + i
print(total)
n = 10
while n > 3:
    n = n - 4
print(n)
```

Answer

10

2

$1 + 2 + 3 + 4 = 10$ (5 is not in).

n: 10, 6, 2. Is 2 > 3? No: stop.



Topic 2

Worked problems

Recitation 2 • Decisions, lists & loops



TODAY

1. Recap and warm-up
2. **Worked problems**
3. Your turn
4. Exam corner
5. Solutions

Problem 1: the campus shawarma deal

The stand's discount rules

Order of 100 ILS or more **and** a student card: 15% off.
Only **one** of the two: 5% off. Neither: full price.

Problem 1: the campus shawarma deal

The stand's discount rules

Order of 100 ILS or more **and** a student card: 15% off.
Only **one** of the two: 5% off. Neither: full price.



Plan in words first:

1. Ask for the total and the student card
2. Both? 15%. At least one? 5%. Else 0
3. Print what to pay

Build it, one step at a time

```
total = float(input("Order total? "))
student = input("Student? (yes/no) ")
```

Build it, one step at a time

```
total = float(input("Order total? "))
student = input("Student? (yes/no) ")
if total >= 100 and student == "yes":
    discount = 0.15
elif total >= 100 or student == "yes":
    discount = 0.05
else:
    discount = 0
```

Build it, one step at a time

```
total = float(input("Order total? "))
student = input("Student? (yes/no) ")
if total >= 100 and student == "yes":
    discount = 0.15
elif total >= 100 or student == "yes":
    discount = 0.05
else:
    discount = 0
pay = total * (1 - discount)
print(f"You pay {pay:.2f} ILS")
```

Build it, one step at a time

```
total = float(input("Order total? "))
student = input("Student? (yes/no) ")
if total >= 100 and student == "yes":
    discount = 0.15
elif total >= 100 or student == "yes":
    discount = 0.05
else:
    discount = 0
pay = total * (1 - discount)
print(f"You pay {pay:.2f} ILS")
```

Run 1

OUTPUT

```
Order total? 120
Student? (yes/no) yes
You pay 102.00 ILS
```

Build it, one step at a time

```python
total = float(input("Order total? "))
student = input("Student? (yes/no) ")
if total >= 100 and student == "yes":
    discount = 0.15
elif total >= 100 or student == "yes":
    discount = 0.05
else:
    discount = 0
pay = total * (1 - discount)
print(f"You pay {pay:.2f} ILS")
```

Run 1

OUTPUT

```
Order total? 120
Student? (yes/no) yes
You pay 102.00 ILS
```

Run 2

OUTPUT

```
Order total? 120
Student? (yes/no) no
You pay 114.00 ILS
```

Run 3 at home: 60 and `no` gives `60.00`.

What if we swap the two checks?

Student, 120 ILS. What is printed?

```
if total >= 100 or student == "yes":
    discount = 0.05
elif total >= 100 and student == "yes":
    discount = 0.15
else:
    discount = 0
```

What if we swap the two checks?

Student, 120 ILS. What is printed?

```
if total >= 100 or student == "yes":
    discount = 0.05
elif total >= 100 and student == "yes":
    discount = 0.15
else:
    discount = 0
```

Answer

You pay `114.00` ILS

The `or` check is True first, so the 15% road is never reached.



Strictest check first!

Problem 2: the exam review sign-up

The TA's sign-up form

Some students clicked "sign up" twice. How many people really come? Sort them A to Z: first 3 go to Room A, the rest to Room B.

Problem 2: the exam review sign-up

The TA's sign-up form

Some students clicked "sign up" twice. How many people really come? Sort them A to Z: first 3 go to Room A, the rest to Room B.



Plan in words:

1. A **set** drops the double clicks
2. Back to a **list**, so we can `sort`
3. **Slice**: `[:3]` and `[3:]`

Code it

review_rooms.py

```
signups = ["Shira", "Itay", "Noa", "Itay",
           "Avi", "Shira", "Maya"]
people = list(set(signups))
people.sort()
print(f"{len(signups)} clicks, {len(people)} people")
print("Room A:", people[:3])
print("Room B:", people[3:])
```

Code it

review_rooms.py

```
signups = ["Shira", "Itay", "Noa", "Itay",
           "Avi", "Shira", "Maya"]
people = list(set(signups))
people.sort()
print(f"{len(signups)} clicks, {len(people)} people")
print("Room A:", people[:3])
print("Room B:", people[3:])
```

OUTPUT

```
7 clicks, 5 people
Room A: ['Avi', 'Itay', 'Maya']
Room B: ['Noa', 'Shira']
```

Code it

review_rooms.py

```
signups = ["Shira", "Itay", "Noa", "Itay",
           "Avi", "Shira", "Maya"]
people = list(set(signups))
people.sort()
print(f"{len(signups)} clicks, {len(people)} people")
print("Room A:", people[:3])
print("Room B:", people[3:])
```

OUTPUT

```
7 clicks, 5 people
Room A: ['Avi', 'Itay', 'Maya']
Room B: ['Noa', 'Shira']
```

`list(...)` turns the set back into a list. Sets have no order, lists can be sorted.

What if there are two review sessions?

Who signed up for both days?

```
sunday = {"Shira", "Itay", "Noa", "Avi"}
tuesday = {"Noa", "Tamar", "Itay", "Yoni"}
both = list(sunday & tuesday)
both.sort()
print(both)
print(len(sunday | tuesday))
```



What if there are two review sessions?

Who signed up for both days?

```
sunday = {"Shira", "Itay", "Noa", "Avi"}
tuesday = {"Noa", "Tamar", "Itay", "Yoni"}
both = list(sunday & tuesday)
both.sort()
print(both)
print(len(sunday | tuesday))
```

Answer

```
['Itay', 'Noa']
6
```

&: in both. |: everyone, once each: $4 + 4 - 2 = 6$.



Problem 3: my screen-time week

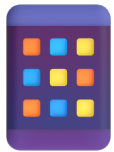
Your phone's weekly report

Minutes per day: 95, 140, 260, 80, 310, 190, 220.
Total? Daily average? How many days over 3 hours?

Problem 3: my screen-time week

Your phone's weekly report

Minutes per day: 95, 140, 260, 80, 310, 190, 220.
Total? Daily average? How many days over 3 hours?



Plan in words:

1. Before: `total = 0`, `heavy_days = 0`
2. Inside: add the minutes, count if over 180
3. After: print total, average, count

Code it: two recipes in one loop

screen_time.py

```
week = [95, 140, 260, 80, 310, 190, 220]
total = 0
heavy_days = 0
for minutes in week:
    total = total + minutes
    if minutes > 180:
        heavy_days = heavy_days + 1
print(f"Total: {total // 60} h {total % 60} min")
print(f"Average: {total / len(week):.0f} min/day")
print(f"Days over 3 hours: {heavy_days}")
```

Code it: two recipes in one loop

screen_time.py

```
week = [95, 140, 260, 80, 310, 190, 220]
total = 0
heavy_days = 0
for minutes in week:
    total = total + minutes
    if minutes > 180:
        heavy_days = heavy_days + 1
print(f"Total: {total // 60} h {total % 60} min")
print(f"Average: {total / len(week):.0f} min/day")
print(f"Days over 3 hours: {heavy_days}")
```

OUTPUT

```
Total: 21 h 35 min
Average: 185 min/day
Days over 3 hours: 4
```

Code it: two recipes in one loop

screen_time.py

```
week = [95, 140, 260, 80, 310, 190, 220]
total = 0
heavy_days = 0
for minutes in week:
    total = total + minutes
    if minutes > 180:
        heavy_days = heavy_days + 1
print(f"Total: {total // 60} h {total % 60} min")
print(f"Average: {total / len(week):.0f} min/day")
print(f"Days over 3 hours: {heavy_days}")
```

OUTPUT

```
Total: 21 h 35 min
Average: 185 min/day
Days over 3 hours: 4
```

`//` and `%` from Lecture 2 turn 1295 minutes into hours and minutes.

What if…? The worst day and a detox plan

The worst day (max recipe)

```
worst = week[0]
for minutes in week:
    if minutes > worst:
        worst = minutes
print(worst, max(week))
```

Cut 20 min a day, every week

```
daily = 185
weeks = 0
while daily >= 120:
    daily = daily - 20
    weeks = weeks + 1
print(f"{weeks} weeks: {daily} min")
```

What if…? The worst day and a detox plan

The worst day (max recipe)

```
worst = week[0]
for minutes in week:
    if minutes > worst:
        worst = minutes
print(worst, max(week))
```

OUTPUT

```
310 310
```

Cut 20 min a day, every week

```
daily = 185
weeks = 0
while daily >= 120:
    daily = daily - 20
    weeks = weeks + 1
print(f"{weeks} weeks: {daily} min")
```

What if...? The worst day and a detox plan

The worst day (max recipe)

```
worst = week[0]
for minutes in week:
    if minutes > worst:
        worst = minutes
print(worst, max(week))
```

OUTPUT

```
310 310
```

Cut 20 min a day, every week

```
daily = 185
weeks = 0
while daily >= 120:
    daily = daily - 20
    weeks = weeks + 1
print(f"{weeks} weeks: {daily} min")
```

OUTPUT

```
4 weeks: 105 min
```

Unknown number of rounds? That is a job for `while`.

Topic 3

Your turn

Recitation 2 • Decisions, lists & loops



TODAY

1. Recap and warm-up
2. Worked problems
3. **Your turn**
4. Exam corner
5. Solutions

Your turn: 10 exercises

Exercise	Lecture	Level	Where
1 Fix the parking sign	L4	★	in class
2 Low battery warning	L4	★★	in class
3 Leap year or not?	L4	★★★	at home
4 Shabbat dinner shopping	L5	★	in class
5 Read the train ticket	L5	★★	in class
6 What should Yoni watch?	L5	★★	at home
7 The even-floor elevator	L6	★	in class
8 Fix the tips average	L6	★★	in class
9 Saving for a festival	L6	★★	at home
10 Password checker	L4-6	★★★	at home



Predict the output *before* you run.
Stuck for 3 minutes? Ask!

Exercise 1 ⭐: fix the parking sign

EXERCISE 1

This code has 3 bugs. Fix them all.

```
hour = 19
day = "Friday"
if day = "Saturday" or hour > 18
    print("Parking is free")
else:
print("Pay at the machine")
```

Wanted output: 🚙

OUTPUT

```
Parking is free
```

Exercise 1 ⭐: fix the parking sign

EXERCISE 1

This code has 3 bugs. Fix them all.

```
hour = 19
day = "Friday"
if day = "Saturday" or hour > 18
    print("Parking is free")
else:
print("Pay at the machine")
```

Wanted output: 🚗

OUTPUT

```
Parking is free
```

Hint

Fix one bug, run, read the **last line** of the error, repeat. Look for =, : and spaces.

Exercise 2 ⭐⭐: low battery warning

EXERCISE 2

fill in the yellow blanks

```
battery = 12
charging = False
if battery < 15 [ ] [ ] charging:
    print("Plug me in NOW!")
[ ] battery < 50:
    print("Okay for now")
[ ]:
    print("All good")
```

Wanted, for each test: 🔋

battery	charging	**prints**
12	False	Plug me in NOW!
12	True	Okay for now
40	False	Okay for now
80	False	All good

Exercise 2 ⭐⭐: low battery warning

EXERCISE 2

fill in the yellow blanks

```
battery = 12
charging = False
if battery < 15 [____] [____] charging:
    print("Plug me in NOW!")
[____] battery < 50:
    print("Okay for now")
[____]:
    print("All good")
```

Wanted, for each test: 🔋

battery	charging	**prints**
12	False	Plug me in NOW!
12	True	Okay for now
40	False	Okay for now
80	False	All good

Hint

Panic only if the battery is low **and** it is **not** charging.

Exercise 3 ⭐⭐⭐: leap year or not?

AT HOME 3

Write it from scratch

Ask for a year. Print if it is a leap year.

- Divisible by 4: leap…
- …but divisible by 100: not leap…
- …but divisible by 400: leap!

Test runs:

Year?	prints
2024	`2024 is a leap year`
1900	`1900 is not a leap year`
2000	`2000 is a leap year`

Exercise 3 ⭐⭐⭐: leap year or not?

AT HOME 3

Write it from scratch

Ask for a year. Print if it is a leap year.

- Divisible by 4: leap…
- …but divisible by 100: not leap…
- …but divisible by 400: leap!

Test runs:

Year?	prints
2024	`2024 is a leap year`
1900	`1900 is not a leap year`
2000	`2000 is a leap year`

Hint

Divisible by 4: `year % 4 == 0`. Check the rarest rule (400) first.

Exercise 4 ⭐: Shabbat dinner shopping

EXERCISE 4

fill in the list methods

```
cart = ["challah", "wine", "hummus"]
cart.        ("salad")
cart.        ("wine")    # we have some
cart.        (0, "candles")
print(cart,      (cart))
```

Wanted output: 🛒

OUTPUT

```
['candles', 'challah', 'hummus', 'salad'] 4
```

Exercise 4 ⭐: Shabbat dinner shopping

EXERCISE 4

fill in the list methods

```
cart = ["challah", "wine", "hummus"]
cart.________("salad")
cart.________("wine")     # we have some
cart.________(0, "candles")
print(cart, ____(cart))
```

Hint

Add to the end, cross off by name, cut in at the front, count.

Wanted output: 🛒

OUTPUT

```
['candles', 'challah', 'hummus', 'salad'] 4
```

Exercise 5 ⭐⭐: read the train ticket

EXERCISE 5

fill in the slices

```
ticket = "TLV-HFA-0830"
origin = ticket[      ]
dest = ticket[      ]
departs = ticket[      ]
print(f"From {origin} to {dest}")
print(f"Leaves at {departs[:2]}:{departs[2:]}")
```

Wanted output: 🚆

OUTPUT

```
From TLV to HFA
Leaves at 08:30
```

Exercise 5 ⭐⭐: read the train ticket

EXERCISE 5

fill in the slices

```
ticket = "TLV-HFA-0830"
origin = ticket[      ]
dest = ticket[      ]
departs = ticket[      ]
print(f"From {origin} to {dest}")
print(f"Leaves at {departs[:2]}:{departs[2:]}")
```

Wanted output: 🚆

OUTPUT

```
From TLV to HFA
Leaves at 08:30
```

Hint

Write the index under each letter: `T`=0, `L`=1, `V`=2, `-`=3… The time is the last 4.

Exercise 6 ⭐⭐: what should Yoni watch?

AT HOME 6

Write it from scratch with sets

Noa: Fauda, Friends, Shtisel, Lupin **Yoni:** Friends, Dark, Fauda, Narcos
Print: how many they share, Noa's tips for Yoni (A to Z), how many shows in total.

Wanted output: 📺

```
OUTPUT
They share 2 shows
For Yoni: ['Lupin', 'Shtisel']
In total: 6 shows
```

Exercise 6 ⭐⭐: what should Yoni watch?

AT HOME 6

Write it from scratch with sets

Noa: Fauda, Friends, Shtisel, Lupin **Yoni:** Friends, Dark, Fauda, Narcos
Print: how many they share, Noa's tips for Yoni (A to Z), how many shows in total.

Wanted output: 📺

```
OUTPUT
They share 2 shows
For Yoni: ['Lupin', 'Shtisel']
In total: 6 shows
```

Hint

`&` both, `-` only mine, `|` all. To sort a set: `list(...)`, then `.sort()`.

Exercise 7 ⭐: the even-floor elevator

EXERCISE 7

stops on even floors only: fill in range

```
for floor in range(   ,    ,    ):
    print(f"Floor {floor}")
print("Top floor!")
```

Wanted output: 🏢

OUTPUT

```
Floor 2
Floor 4
Floor 6
Floor 8
Floor 10
Top floor!
```

Exercise 7 ⭐: the even-floor elevator

EXERCISE 7

stops on even floors only: fill in range

```
for floor in range(   ,    ,    ):
    print(f"Floor {floor}")
print("Top floor!")
```

Wanted output: 🏢

OUTPUT

```
Floor 2
Floor 4
Floor 6
Floor 8
Floor 10
Top floor!
```

Hint

`range(start, stop, step)`: the stop is **never** in. Which number comes after 10?

Exercise 8 ⭐⭐: fix the tips average

EXERCISE 8

No error, but 3 bugs. Fix them.

```
tips = [20, 35, 0, 50, 15]
for tip in tips:
    total = 0
    total = total + tip
    print(f"Average tip: {total / 4}")
```

Wanted output: 🪙

OUTPUT

```
Average tip: 24.0
```

Exercise 8 ⭐⭐: fix the tips average

EXERCISE 8

No error, but 3 bugs. Fix them.

```
tips = [20, 35, 0, 50, 15]
for tip in tips:
    total = 0
    total = total + tip
    print(f"Average tip: {total / 4}")
```

Wanted output: 🪙

OUTPUT

```
Average tip: 24.0
```

Hint

Run it: 5 lines, not 1. What should run **once**, before or after the loop? How many tips are there?

Exercise 9 ⭐⭐: saving for a festival

AT HOME 9

✏️ Write it from scratch with while

Ticket: **450 ILS**. You have **150 ILS** and save **75 ILS** a week. Print the balance every week, then the number of weeks.

Wanted output: 🎫

OUTPUT

```
Week 1: 225 ILS
Week 2: 300 ILS
Week 3: 375 ILS
Week 4: 450 ILS
Ticket after 4 weeks!
```

Exercise 9 ⭐⭐: saving for a festival

AT HOME 9

Write it from scratch with while

Ticket: **450 ILS**. You have **150 ILS** and save **75 ILS** a week. Print the balance every week, then the number of weeks.

Hint

Weeks unknown in advance: `while`. Two boxes change each round.

Wanted output: 🎫

OUTPUT

```
Week 1: 225 ILS
Week 2: 300 ILS
Week 3: 375 ILS
Week 4: 450 ILS
Ticket after 4 weeks!
```

Exercise 10 ⭐⭐⭐: password checker

AT HOME 10

Write it from scratch

Ask for a password, count its digits with a loop. **Strong**: 8+ characters **and** 2+ digits.

Test: Password?	prints first	then
sunshine	Digits: 0	Weak password
telaviv2026	Digits: 4	Strong password!
ab12	Digits: 2	Weak password

Exercise 10 ⭐⭐⭐: password checker

AT HOME 10

Write it from scratch

Ask for a password, count its digits with a loop. **Strong**: 8+ characters **and** 2+ digits.

Test: Password?	prints first	then
sunshine	Digits: 0	Weak password
telaviv2026	Digits: 4	Strong password!
ab12	Digits: 2	Weak password

Hint

`in` works on text too: `"5" in "0123456789"` is `True`.

Your home practice list

Before the next recitation

- Exercises 3, 6, 9 and 10 **at home**
- Any in-class exercise you did not finish
- Then check yourself against the solutions

Type the code yourself. Reading a solution is not practice.



Topic 4

Exam corner

Recitation 2 • Decisions, lists & loops



TODAY

1. Recap and warm-up
2. Worked problems
3. Your turn
4. **Exam corner**
5. Solutions

Exam style 1: trace the loop

What is printed?

```
nums = [3, 8, 5, 12, 7]
result = 0
for num in nums:
    if num % 2 == 0:
        result = result + num
    else:
        result = result - 1
print(result)
```



Exam style 1: trace the loop

What is printed?

```
nums = [3, 8, 5, 12, 7]
result = 0
for num in nums:
    if num % 2 == 0:
        result = result + num
    else:
        result = result - 1
print(result)
```

Answer

17

num	3	8	5	12	7
result	-1	7	6	18	17

Even: add it. Odd: take 1 away.



Exam tip: one column per round. Write the new value of every box.

Exam style 2: no error, wrong answer

Should print 15. Find the bug.

```
total = 0
i = 1
while i < 5:
    total = total + i
    i = i + 1
print(total)
```

OUTPUT

```
10
```

Exam style 2: no error, wrong answer

Should print 15. Find the bug.

```
total = 0
i = 1
while i < 5:
    total = total + i
    i = i + 1
print(total)
```

OUTPUT

```
10
```

Fix: <= instead of <

```
while i <= 5:
```

OUTPUT

```
15
```

We want $1 + 2 + 3 + 4 + 5$. When `i` is 5, `5 < 5` is False, so 5 was never added. Classic “off by one”.

Exam style 3: slices and bits

What is printed? Write every line.

```
word = "Mississippi"
print(word[:4])
print(word[-4:])
print(word[::3])
print(bin(len(word)))
```



Exam style 3: slices and bits

What is printed? Write every line.

```
word = "Mississippi"
print(word[:4])
print(word[-4:])
print(word[::3])
print(bin(len(word)))
```



Answer

```
Miss
ippi
Mssp
0b1011
```

`[::3]` takes positions 0, 3, 6, 9.
The word has 11 letters: $8 + 2 + 1$.

Three new mistakes to avoid

```
if grade >= 56:
    print("Passed")
else grade < 56:
    print("Failed")
```

ERROR

```
SyntaxError:
expected ':'
```

`else` takes **no** condition. It means “everything else”.

```
bag = ["pita"]
bag.append("tea", "jam")
```

ERROR

```
TypeError:
list.append() takes
exactly one argument
(2 given)
```

One item per `append`. Call it twice.

```
reply = "n"
if reply == "y" or "Y":
    print("Order sent!")
```

OUTPUT

```
Order sent!
```

No error, wrong road! `"Y"` alone counts as True. Ask in full: `reply == "Y"`

Before the homework

⚠ Next: HW2, Functions & loops

Conditions, loops, lists, sets, strings and basic functions.
Opens after Lecture 7, submit on Moodle by ∼**Dec 20**.





- Test every road of every `if`
- Loops: write “before / inside / after” first

Recap

✔ Today you learned

- `if`/`elif`/`else`: strictest check first, first True wins
- Lists keep order, sets drop duplicates, slices stop *before* stop
- Loops: set up before, update inside, use after



Topic 5

Solutions

Recitation 2 • Decisions, lists & loops



TODAY

1. Recap and warm-up
2. Worked problems
3. Your turn
4. Exam corner
5. **Solutions**

Solution 1: fix the parking sign

SOLUTION

```
hour = 19
day = "Friday"
if day == "Saturday" or hour > 18:
    print("Parking is free")
else:
    print("Pay at the machine")
```

OUTPUT

```
Parking is free
```

- == to ask, not =
- A colon : after the condition
- Indent the **print** under **else** by 4 spaces

Friday is not Saturday, but 19 > 18 is True. or needs just one.

Solution 2: low battery warning

SOLUTION

```
battery = 12
charging = False
if battery < 15 and not charging:
    print("Plug me in NOW!")
elif battery < 50:
    print("Okay for now")
else:
    print("All good")
```

OUTPUT

```
Plug me in NOW!
```

With `charging = True`, `not charging` is False, so the first check fails. `12 < 50` is True: `Okay for now`.



Solution 3: leap year or not?

SOLUTION

```
year = int(input("Year? "))
if year % 400 == 0:
    print(f"{year} is a leap year")
elif year % 100 == 0:
    print(f"{year} is not a leap year")
elif year % 4 == 0:
    print(f"{year} is a leap year")
else:
    print(f"{year} is not a leap year")
```

OUTPUT

```
Year? 1900
1900 is not a leap year
```

OUTPUT

```
Year? 2000
2000 is a leap year
```

Rarest rule first. Swap the order and 2000 comes out wrong!

Solutions 4 and 5: shopping and the ticket

SOLUTION

Exercise 4

```
cart = ["challah", "wine", "hummus"]
cart.append("salad")
cart.remove("wine")
cart.insert(0, "candles")
print(cart, len(cart))
```

Output: `['candles', 'challah', 'hummus', 'salad'] 4`

Exercise 5

```
ticket = "TLV-HFA-0830"
origin = ticket[ :3 ]
dest = ticket[ 4:7 ]
departs = ticket[ -4: ]
```

`HFA` sits at 4, 5, 6, so stop is 7. `[8:]` also works for the time.

Solution 6: what should Yoni watch?

SOLUTION

```
noa = {"Fauda", "Friends", "Shtisel", "Lupin"}
yoni = {"Friends", "Dark", "Fauda", "Narcos"}
print(f"They share {len(noa & yoni)} shows")
for_yoni = list(noa - yoni)
for_yoni.sort()
print(f"For Yoni: {for_yoni}")
print(f"In total: {len(noa | yoni)} shows")
```

OUTPUT

```
They share 2 shows
For Yoni: ['Lupin', 'Shtisel']
In total: 6 shows
```

Sorting first means the output is the same on every computer. Sets alone have no order.

Solution 7: the even-floor elevator

SOLUTION

```
for floor in range( 2 , 11 , 2 ):
    print(f"Floor {floor}")
print("Top floor!")
```

Stop must be **past** 10: 11 or 12 both work. `range(2, 10, 2)` would forget floor 10.

OUTPUT

```
Floor 2
Floor 4
Floor 6
Floor 8
Floor 10
Top floor!
```

Solution 8: fix the tips average

SOLUTION

```
tips = [20, 35, 0, 50, 15]
total = 0
for tip in tips:
    total = total + tip
print(f"Average tip: {total / len(tips)}")
```

OUTPUT

```
Average tip: 24.0
```

- `total = 0` goes **before** the loop
- `print` goes **after**: no indent
- Divide by `len(tips)`: 5, not 4

The buggy version printed 5.0, 8.75, 0.0, 12.5, 3.75.

Solution 9: saving for a festival

SOLUTION

```
savings = 150
weeks = 0
while savings < 450:
    savings = savings + 75
    weeks = weeks + 1
    print(f"Week {weeks}: {savings} ILS")
print(f"Ticket after {weeks} weeks!")
```

OUTPUT

```
Week 1: 225 ILS
Week 2: 300 ILS
Week 3: 375 ILS
Week 4: 450 ILS
Ticket after 4 weeks!
```

450 < 450 is False, so the loop stops at exactly 450.

Solution 10: password checker

SOLUTION

```
password = input("Password? ")
digits = 0
for letter in password:
    if letter in "0123456789":
        digits = digits + 1
print(f"Digits: {digits}")
if len(password) >= 8 and digits >= 2:
    print("Strong password!")
else:
    print("Weak password")
```

Three lectures in one program!

OUTPUT

```
Password? telaviv2026
Digits: 4
Strong password!
```

OUTPUT

```
Password? ab12
Digits: 2
Weak password
```

3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md